Lesson 19 – Data Logging and Drive Methods Update

In this lesson you will continue to work on your branch of the RomBot2026 code.

In lesson 16 you set up manual drive methods, so for review you will get your Romi back to driving with the Xboxcontroller.

You will also learn about logging.

Important: Update your branch from Main.

Exercise 1

Task: Create a command DriveStraightDistance and add a Widget to Elastic to start the command.

Task: Create DriveWithJoystick command.

Task: Make DriveWithJoystick the default command for the romiDrivetrain subsystem.

Data Logging:

The data logging methods are in the DataLogUtil utility class.

When we want to add text to the log file we will use the writeMessage method or the writeMessageEcho method. Both of these append data to the open log file with the latter also sending the data to the Terminal. Look at the RobotContainer code to see how logging is used. Every time the system state is changed (ie. teleop to disabled, etc.) the log file is appended to show the time that the change occurred. The method writeMessageEcho is used so the terminal is also updated to show the new state.

```
/**
  * Method called when autonomouse mode is initialized/enabled.
  */.
T
 public void autonomousInit() {
   DataLogUtil.writeMessageEcho("Auto: Mode Init ");
 }
```

Change from autonomous to disabled and look at the Terminal to see these printouts. The first time you run this, a new logfile is opening with the name showing the date and time.

Exercise 2

Task: Add data logging to DriveTime in initialize().

Task: Add Widgets to Elastic to display percent setting and velocity for each motor.

(This data is sent to the SmartDashboard in RomiDriveTrain)

Task: Test and debug.

Task: Commit all code and push to remote!!

We mentioned that this data is so important to be able to analyze the robot performance for auto routines and during the match. More important than just text, is the full logging of many robot parameters that might be needed. This is explained in more detail in the slides 34-38 from Don Sawdai's programming overview.

https://drive.google.com/file/d/1JjOCK-y2pyeggV7KtjM_QNYnRqZBqc5z/view?usp=drive_link

Logging Messages to the DataLog

Logging data to the DataLog is critical for calibrating robots and for debugging robot behavior

Start the DataLogUtil in Robot.java

- Start the DataLogUtil *before* creating the RobotContainer

```
28   public void robotInit() {
29     // Start Datalogger first!
30     DataLogUtil.start();
31
32     // Instantiate our RobotContainer.  This wi
33     // autonomous chooser on the dashboard.
34     m_robotContainer = new RobotContainer();
35     m_robotContainer.robotInit();
36   }
```

Use writeMessage() or writeMessageEcho() to write a *string* to the DataLog

- This is useful for logging when events occur (command Init, subsystem calibrated, etc)
- The messages will appear in the DataLog file under the LogEntry key "/DataLog/message"
- writeMessageEcho() will also print the data to the Driver Station console window
- Do not use writeMessage for *data that you want to plot* [see next slide for logging *data* instead]
- Do not use the + operator to concatenate strings!!! It is very slow and will delay code.

```
public RobotContainer() {
  DataLogUtil.writeMessageEcho("RobotContainer: Constructor, Version =", Constants.bcrRobotCodeVersion);
```

Start the message with the class and method names. That makes messages easier to sort and find.

- Strings and data to write to log
- All data will be converted to strings
- All parameters will be concatenated together with one space added between each parameter

Logging Data to the DataLog

Each data element should be logged to its own LogEntry (key) in the DataLog

- LogEntries have a hierarchy. Recommended key names:
  - */SubsystemName/DataElementName*
  - */CommandName/DataElementName*

Define LogEntries with keys as *final* member variables in the subsystem or command

- Basic key types are Double, Boolean, and String
- Compound key types include DoubleArray, BooleanArray, StringArray, and Struct

Get DataLog object

Create LogEntries

```
private final DataLog log = DataLogManager.getLog();
private final StructLogEntry<Pose2d> dLogOdometryPose2D = StructLogEntry.create(log, "/RomiDrivetrain/curPose2d", Pose2d.struct);
private final DoubleLogEntry dLogLeftPercent = new DoubleLogEntry(log, "/RomiDrivetrain/LeftPercent");
private final DoubleLogEntry dLogRightPercent = new DoubleLogEntry(log, "/RomiDrivetrain/RightPercent");
private final BooleanLogEntry dLogRobotMoving = new BooleanLogEntry(log, "/RomiDrivetrain/RobotIsMoving");
private final StringLogEntry dLogDrivetrainColor = new StringLogEntry(log, "/RomiDrivetrain/Color");
```

Record data to the Log periodically or when relevant using LogEntry.append()

- .append() has *optional* 2nd parameter to specify the timestamp for the data.
  This is very useful to align multiple parameters vs time.

Get current timestamp

Record data to LogEntries

```
long timeNow = RobotController.getFPGATime();

dLogOdometryPose2D.append(getPose(), timeNow);
dLogLeftPercent.append(getLeftPercent(), timeNow);
dLogRightPercent.append(getRightPercent(), timeNow);
```

Logging Data to the DataLog – Subsystem Example

Subsystem Class level:

- Implement Loggable and create variables to track fast (every 20 msec) vs. slow (every 200 msec) logging
- Define LogEntry member variables with keys

```
public class RomiDrivetrain extends SubsystemBase implements Loggable {
  // DataLog variables
  private int logRotationKey;          // key for the logging cycle for this subsystem
  private boolean fastLogging = false; // true is enabled to run every cycle; false follows normal logging cycles
  private final DataLog log = DataLogManager.getLog();
  private final StructLogEntry<Pose2d> dLogOdometryPose2D = StructLogEntry.create(log, "/RomiDrivetrain/curPose2d", P
  private final DoubleLogEntry dLogLeftPercent = new DoubleLogEntry(log, "/RomiDrivetrain/LeftPercent");
  private final DoubleLogEntry dLogRightPercent = new DoubleLogEntry(log, "/RomiDrivetrain/RightPercent");
```

Subsystem constructor:

- Allocate a logRotationKey (needed for slow logging)
- Prime the DataLog with initial data (or dummy data). This will allocate memory/objects at robot boot time, which will prevent a short delay when the robot is first enabled
- *Optional*: writeMessage() with boot info, calibration, etc.

```
public RomiDrivetrain() {
  logRotationKey = DataLogUtil.allocateLogRotation();     // Get log rotation for this subsystem

  // Initialize the drivetrain hardware
  // [etc...]

  // Prime the DataLog to reduce delay when first enabling the robot
  updateLog(true);
}
```

Subsystem method that logs the data

- Option to only log data if the robot is enabled (<file size)

```
/**
 * Write information about the hopper to the file log.
 * @param logWhenDisabled true = write when robot is disabled, false = only write when robot is enabled
 */
public void updateLog(boolean logWhenDisabled) {
  if (logWhenDisabled || !DriverStation.isDisabled()) {
    long timeNow = RobotController.getFPGATime();

    dLogOdometryPose2D.append(getPose(), timeNow);
    dLogLeftPercent.append(getLeftPercent(), timeNow);
    dLogRightPercent.append(getRightPercent(), timeNow);
  }
}
```

Subsystem periodic():

- Log important data for the subsystem
- Option to only log data fast (every 20 msec, use rarely) or slow (every 200 msec, should be the default)

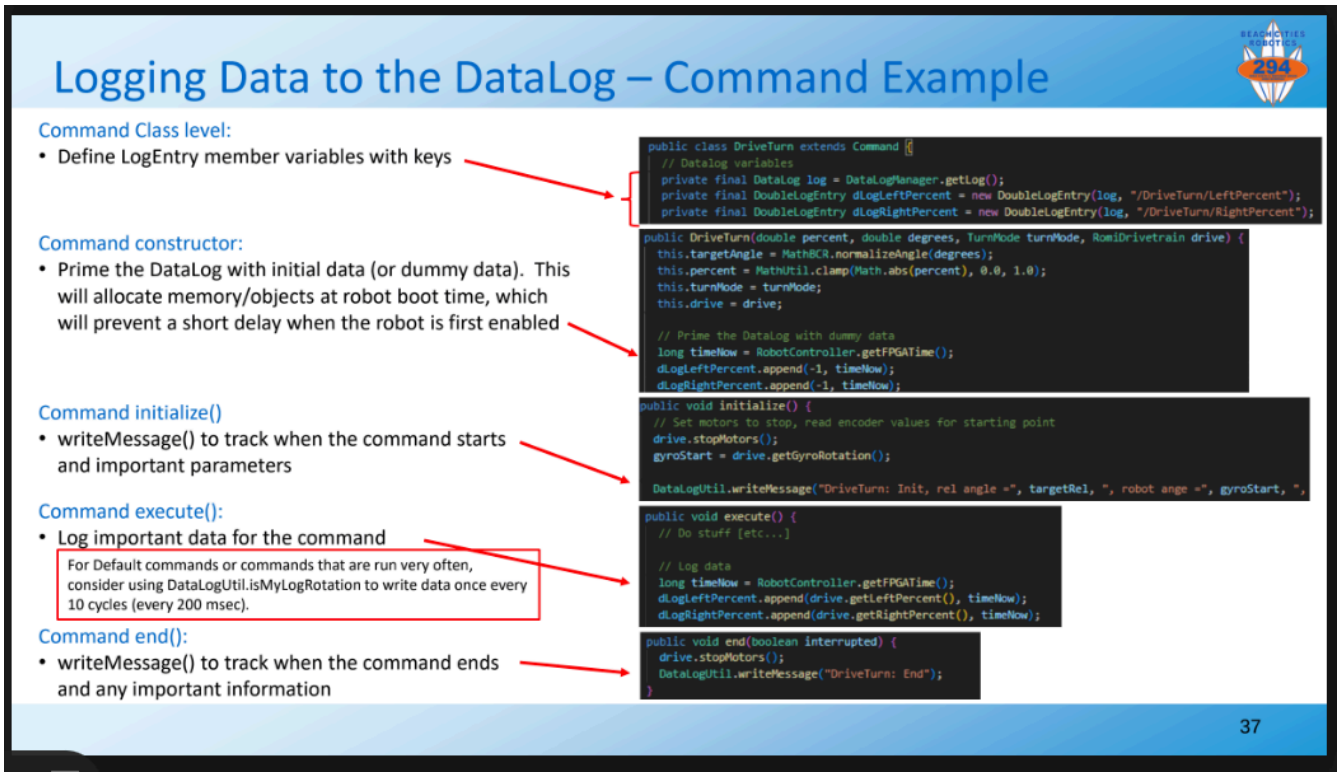
```
@Override
public void periodic() {
  // This method will be called once per scheduler run

  // Update robot odometry
  // [etc...]

  if(fastLogging || DataLogUtil.isMyLogRotation(logRotationKey)) {
    updateLog(false);
  }

  if(DataLogUtil.isMyLogRotation(logRotationKey)) {
    // Update data on SmartDashboard
    SmartDashboard.putNumber("Pose X", curPose.getX());
```

- Don't write to logfiles faster than needed!!!!
- Use DataLogUtil.isMyLogRotation to write data once every 10 cycles (every 200 msec), which is often enough data



We have to control how often we log data and how much data is logged. The writing of the data to the file takes time and can use valuable system resources which can degrade the actual performance if too excessive. If we keep adding data and don't delete old files, we can run out of onboard memory space and crash the program. That is why you will see log rotation and levels of logging to control these resources. When debugging, you may want to record extra data, but during a competition match, reduce the logging to just what is essential for evaluation.